

```
1 # Check Point 4 - (CP1 2º Semestre de 2026)
2
3     Nome Completo:
4     RA:
5
6 -----
7
8
9 ## COMO SALVAR O ARQUIVO:
10
11     Use o seu primeiro nome, seguido de '_' com seu último nome, novamente '_' e seu RA.
12     A extensão deste arquivo é '.zip'
13     Exemplo:
14
15     Fulano Cricrano de Souza
16     RA 573312
17
18     fulano_souza_573312.zip
19
20     O mesmo para o/s arquivos extras e/ou que acharem pertinentes.
21
22     **Façam a compactação de todos em um único arquivo
23     e subam no ambiente que foi solicitado!**
24
25 -----
26 ## CP4 - Exercício 1
27
28 Crie um módulo, chamado 'checaCadastroPet.py', que contenha os seguintes elementos:
29
30
31 ### 1-a) **(15 Pontos)**
32
33     Como entrada, a função deve receber:
34     - uma dicionário de listas (use a dica/"hint" para o parâmetro conforme visto em
35       sala)
36     - A STRING com ID do Pet;
37     - Uma lista com nome, tipo e ano de nascimento (estes não serão testados) como
38       no exemplo:
39       ["Lulu", "Cachorro", 2020]
40
41     Uma função que faça a verificação do número de identificação (ID) do PET, sendo que,
42     este identificador deve ter 11 caracteres e a seguinte lógica:
43
44     * Os 2 primeiros dígitos devem ser do tipo de animal:
45       - CC para cães;
46       - GT para gatos;
47       - MM para demais mamíferos;
48       - AV para aves no geral;
49       - RP para reptéis no geral;
50       - OT para outros.
51
52     * Os 5 dígitos subsequentes são o REGISTRO sequencial (começa em 1 e não em 0);
53
54     * os demais 4 dígitos devem ser o ano do nascimento do PET (exemplo 2024).
55
56     O ID recebido deve ser no assim (formato 1):
57
58     'TT-IIIII-AAAA'
59
60     Mas deve aceitar entradas sem os hífen (formato 2):
61
62     'TTIIIIIAAAA'
63
64     ** E rejeitar quaisquer outros tipos de entrada!!! **
65
66     - Primeiro checar o tamanho da string de entrada;
```

```

66
67         - Primeira dica é tratar os dados de entrada começando pela diferença entre
        os formatos 1 e 2;
68
69         - Checar o tipo usando uma lista de referência;
70
71         - Depois checar o registro sequencial como número e se está no item
        correspondente das chaves
72             do dicionário de entrada
73             - Se estiver recuse;
74
75         - Depois checar a data, como número.
76
77
78     Devem INDICAR os parâmetros de entrada da função (como já dito acima), bem como o
        tipo e saída!
79
80
81     Devem inserir o texto descritivo da função (def checaCadastroPET ...), conforme
        explicado em sala!
82
83
84     A SAÍDA (RETORNO) DEVE SER o proprio dicionário que entrou:
85         - Acrescido com o elemento se passar em todo o teste
86         - Ou do mesmo jeito que entrou no caso contrário
87
88
89
90     ### 1-b) **(5 Pontos)**
91
92     Crie um bloco de teste dentro do módulo, usando laço condicional e a variável
        "__name__"
93     para testar a função usando a lista abaixo e dois IDs fornecidos abaixo:
94
95         db_pets = {
96             ("CC","00001",2024):
97                 [
98                     'Max',
99                     'Cachorro',
100                    2024
101                ],
102             ("GT","00002",2018):
103                 [
104                     'Miau',
105                     'Gato',
106                     2018
107                ],
108             }
109
110     cadastroTeste1 = ("CC","00001",2023):
111         [
112             'Lulu',
113             'Cachorro',
114             2020
115         ]
116
117     cadastroTeste2 = ("AV","00003",2025):
118         [
119             'Loro',
120             'Ave',
121             2025
122         ]
123
124     O teste deve rodar sempre que o módulo for executado por si só, e nunca quando
        importado.
125
126
127
128     -----

```

```

129
130 ## CP4 - Exercício 2
131
132 Crie um módulo, chamado 'cadastroPet.py', que contenha os seguintes elementos:
133
134
135 ### 2-a) *(8 pontos)*
136
137 Criar uma função (def petCreate) para criar um cadastro de pets onde:
138
139 Nos parâmetros de entrada, deve inserir:
140
141     - O dicionário atual de PETS cadastrados;
142     - O tipo do PET (Por extenso: "Cachorro");
143     - O ano de nascimento do PET;
144     - O nome do PET.
145
146 A lista de PETS cadastrados deve ser um dicionário de listas, e este deve ser
147 atualizado e devolvida com o **RETURN** no programa;
148
149 Exemplo dde dados está no arquivo petsDB.py
150
151 - Quando executar o programa, o ID deve ser atribuído automaticamente, levando em
152 conta que ele é sequencial!!!
153     Lembrem dos métodos para dicionários para facilitar!!
154
155 - Deve também identificar o tipo do PET (dê preferência para usar uma extrutura de
156 decisão do tipo mach/case, e não perca tempo com maiúsculas e minúsculas, deixe
157 tudo em 'lower' case)
158
159 USEM A FUNÇÃO " ChecaCadastroPest" ANTES DO "RETURN".
160
161 Devem definir os parâmetros de entrada da função, que serão do tipos pertinentes,
162 bem como o tipo e saída!
163
164 Devem inserir o texto descritivo da função (def cadastroPET ...), conforme explicado
165 em sala!
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188

```

2-b) *(8 pontos)*

Crie um procedimento (def petRead) que, ao receber a lista de cadastro de PETS, e o ID de um PET, faça a impressão dos dados de um PET cadastrado, buscando ele na lista de dicionários, pelo "ID":

```

* print('Dados do seu PET: ... \n')
* print(f 'Nome: {nome} \n')
* print(f'Ano de nascimento: {ano} \n')
* print(f'Raça: {tipo}')
* print(f'ID: {id}')

```

Para casos sem erros.

E:

```

* print('O ID no PET não foi encontrado no cadastro \n')
* print(f'Confira os dados e tente novamente!')

```

Considerando que podem haver dois pets com mesmo nome, pensem em usar listas ou outras estruturas para construir a saída.

Devem definir os parâmetros de entrada da função, que serão do tipos pertinentes, bem como o tipo e saída!

Devem inserir o texto descritivo da função (def petRead ...), conforme explicado em sala!

```

189
190
191
192 ### 2-c) *(8 pontos)*
193
194     Crie uma função (def petDelete) que, ao receber a o cadastro de PETS (dicionário), e
        o ID de um PET, busque o PET no dicionário e faça a remoção do cadastro, RETORNANDO
        o dicionário sem o elemento.
195
196     Deve dar os avisos pertinentes:
197
198         print('O ID no PET não foi encontrado no cadastro \n')
199         print(f'Confira os dados e tente novamente!')
200
201
202     Para os casos onde o elemento não esteja no cadastro, e:
203
204         print(f'Cadastro do PET id: {id}, removido com sucesso!')
205
206
207     Devem definir os parâmetros de entrada da função, que serão do tipos pertinentes,
        bem como o tipo e saída!
208
209     Devem inserir o texto descritivo da função (def petDelete ...), conforme explicado
        em sala!
210
211
212 ### 2-d) *(8 pontos)*
213
214     Crie uma função (def petUpdate) que, ao receber a o cadastro de PETS (dicionário), e
        o ID de um PET, busque o PET no dicionário e faça a ATUALIZAÇÃO do cadastro,
        RETORNANDO o dicionário.
215
216     Deve dar os avisos pertinentes:
217
218         print('O ID no PET não foi encontrado no cadastro \n')
219         print(f'Confira os dados e tente novamente!')
220
221
222     Para os casos onde o elemento não esteja no cadastro, e:
223
224         print(f'Cadastro do PET id: {id}, atualizado com sucesso!')
225
226     - O ID não pode ser corrigido, por motivos óbvios!
227
228     Abra um campo de correção para cada elemento da lista e se:
229     - Nada for digitado no campo, não substitua.
230     - Caso o tipo não esteja na lista esperada, não corrija
231     - Se o ano não for um número, não corrija
232
233     - Dê uma mensagem das substituições, com o que estava para o novo:
234         " O nome 'Lulu' foi substituído para 'Zeze'! "
235
236
237     Devem definir os parâmetros de entrada da função, que serão do tipos pertinentes,
        bem como o tipo e saída!
238
239     Devem inserir o texto descritivo da função (def petUpdate ...), conforme explicado
        em sala!
240
241
242 ### 2-e) *(8 pontos)*
243
244     Crie um procedimento que liste, 1 a 1, os PETS que estão no dicionário de PETS (def
        petList).
245     Use a melhor formatação que achar pertinente
246
247     Devem definir os parâmetros de entrada da função, que serão do tipos pertinentes, bem
        como o tipo e saída!

```

```

248
249     Devem inserir o texto descritivo da função (def petDelete ...), conforme explicado
    em sala!
250
251
252
253 ### 2-f) **(10 pontos)**
254
255     Explique a diferença entre **funções** e **procedimentos** vistos em aula:
256
257     RESPONDA AQUI MESMO, ou como texto no seu arquivo cadastroPET.py
258
259
260
261
262
263 ### 2-g) **(10 pontos)**
264
265     Crie um bloco de teste dentro do módulo, usando laço condicional e a variável
    "__name__"
266     para testar a função usando a lista abaixo:
267
268         db_pets = {
269             ("CC", "00001", 2024):
270                 [
271                     'Max',
272                     'Cachorro',
273                     2024
274                 ],
275             ("GT", "00002", 2018):
276                 [
277                     'Miau',
278                     'Gato',
279                     2018
280                 ],
281         }
282
283     cadastroTeste1 = ("CC", "00001", 2023):
284         [
285             'Lulu',
286             'Cachorro',
287             2020
288         ]
289
290
291
292     O teste deve rodar sempre que o módulo for executado por si só, e nunca quando
    importado.
293
294     Faça um teste para cada função (2-a até 2-e) e use os seguintes dados:
295
296     print('Executando Testes das Funções!!! \n')
297     print(50*'-')
298
299     Para 2-a:
300         petCreate(cadastroTeste1, 'OT', 2022 , 'TESTE')
301         print(50*'-')
302
303     Para 2-b:
304         petRead(cadastroTeste1, "Max")
305         print(50*'-')
306
307     Para 2-c:
308         petDelete(cadastroTeste1, "00003")
309         print(50*'-')
310
311     Para 2-d:
312         petUpdate(cadastroTeste1, "00002")
313         faça update do campo ano para 2019

```

```
314         print(50*'-' )
315
316     Para 2-e:
317     print(50*'-' )
318
319
320     -----
321
322     ## CP4 - Exercício 3
323
324     ### 3-a) **(25 pontos)**
325
326     Crie um módulo, chamado 'principal.py', que contenha os seguintes elementos:
327
328
329     - Importe os demais módulos que forem pertinentes, incluindo os dados!
330
331     - Crie um menu que contenha as seguintes opções:
332
333         - Listar todos os PETS;
334         - Buscar PET;
335         - Cadastrar PET;
336         - Descadastrar PET;
337         - Atualizar PET;
338         - Sair
339
340     - Use numeros para selecionar os itens;
341
342     - Use o bloco Try/Except para lidar com os problemas de entrada da função de
    cadastro!!!
343
344
345     Boa prova a todos!!!
346
```